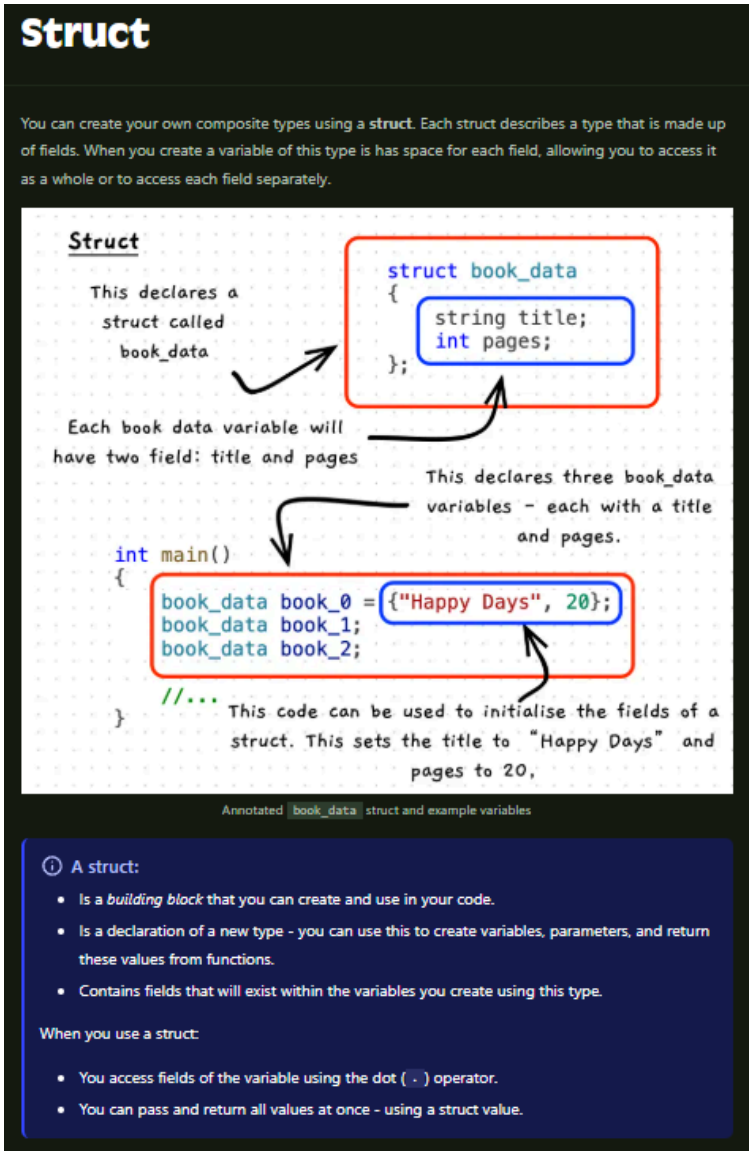
Organising Data

Name: Rahul Vedant Pemsing
Student ID: 30365529
Date: 25/08/2025

Learning Journey and Evidence

To study and master this topic I did the following:

24/08/2025 - Read field guide[Part 2->5.Working with multiple]

I had prior knowledge for both the theory and coding for structure as it was required for my A-level. So reading the field was a good refresher to my knowledge.

Struct

You can create your own composite types using a **struct**. Each struct describes a type that is made up of fields. When you create a variable of this type is has space for each field, allowing you to access it as a whole or to access each field separately.

Struct

This declares a struct called book_data

```
struct book_data
{
    string title;
    int pages;
};
```

Each book data variable will have two field: title and pages

This declares three book_data variables - each with a title and pages.

```
int main()
{
    book_data book_0 = {"Happy Days", 20};
    book_data book_1;
    book_data book_2;

    //...
}
```

This code can be used to initialise the fields of a struct. This sets the title to "Happy Days" and pages to 20,

Annotated `book_data` struct and example variables

A struct:

- Is a *building block* that you can create and use in your code.
- Is a declaration of a new type - you can use this to create variables, parameters, and return these values from functions.
- Contains fields that will exist within the variables you create using this type.

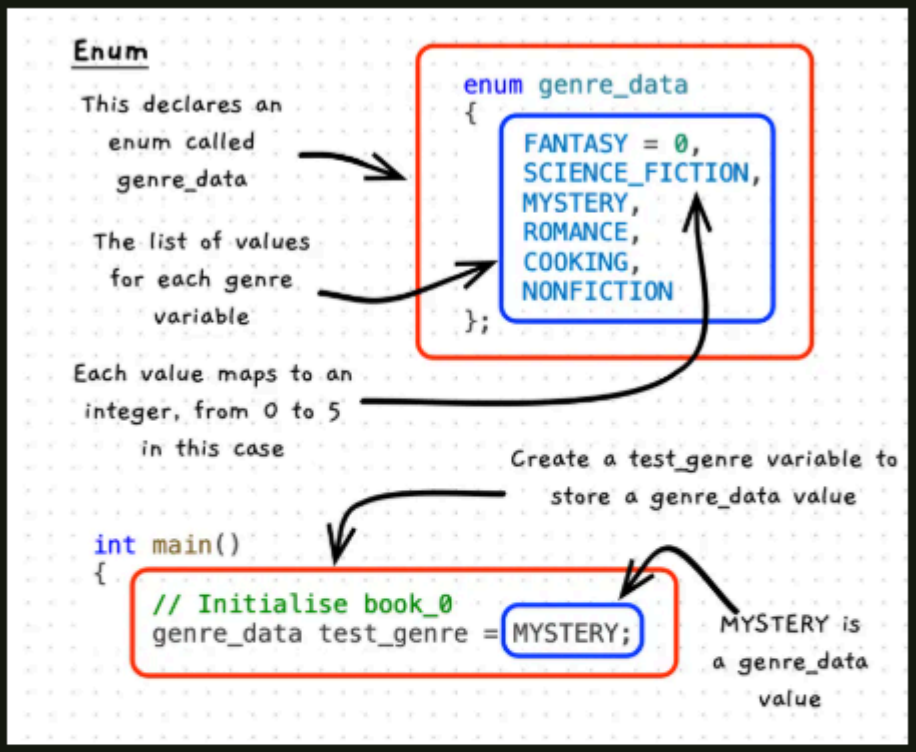
When you use a struct:

- You access fields of the variable using the dot (`.`) operator.
- You can pass and return all values at once - using a struct value.

24/08/2025 - Read field guide[Part 2->3.Structuring data]

Enum

An **enum** is another type you can create. This gives you the ability to define a list of values that the type accepts.



Annotated `genre_data` enum and example variables

An enum:

- Is a *building block* that you can create and use in your code.
- Is a declaration of a new type - you can use this to create variables, parameters, and return these values from functions.
- Lists the values that the type accepts.
- Is actually stored as an integer - starting at 0 for the first value.
- Can use a type cast to convert to and from an integer value.

When you use an enum, you can:

- Use one of the names you declare in the type.
- Convert it to an integer or from an integer when needed.
- Use a switch statement with branches for the different values.

Like structure, I had prior knowledge of this but only for the theory, i never used it in any of my past programs. So reading the field helped me refresh my knowledge for the theory section and learned how to implement it in actual programs.

24/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Guided Tour → Explore enums]

Explore Enums

To get started with structuring data, explore working with an enumeration.

Days

Enumerations allow you to capture a list of options within a program. To test this out let's create an enumeration to represent days of the week. We can call this enumeration `day`, and it can have options for Monday, Tuesday, etc.

To test this out create a new C++ program and declare within this the new enumeration. For the moment, we need to declare the new type and use it to create a variable in `main`.

Complete the following code by declaring the `day` enumeration, and ensuring it has a value for each day. Remember that these values are like constants, so in C/C++ our coding convention would be to use UPPER_CASE. We can make use of our utilities to help with reading details from the user.

Code:

```
#include "splashkit.h"
#include "utilities.h"

using std::to_string;

const int NUM_DAYS = 7;

enum day
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};

day read_day(string prompt)
{
    int day_number;
    write_line(prompt);

    for (int i =0; i< NUM_DAYS;i++)
    {
        day current_day = (day)i;
        write_line(to_string(i+1)+ ": " + to_string(current_day));
    }
    day_number= read_integer("Enter a day number (1-7): ")-1;
    return (day)day_number;
}
```

```
string to_string(day d)
{
    switch(d)
    {
        case MONDAY:
        return "Monday";
        case TUESDAY:
        return "Tuesday";
        case WEDNESDAY:
        return "wednesday";
        case THURSDAY:
        return "Thursday";
        case FRIDAY:
        return "Friday";
        case SATURDAY:
        return "Satuday";
        case SUNDAY:
        return "Sunday";
        default:
        return "Invalid day";
    }
}

int main()
{
    day today;
    today = read_day("what day is it today? ");

    write_line("Today is day " + to_string(today));
    return 0;
}
```

Output:

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/explore_enums_days
$ clang++ enums_days.cpp utilities.cpp -o enums -l splashkit

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/explore_enums_days
$ ./enums
what day is it today?
1: 0
2: 1
3: 2
4: 3
5: 4
6: 5
7: 6
Enter a day number (1-7): 2
Today is day Tuesday
```

By working through the enums exercise, I learned how enumerations restrict a variable to a fixed set of values. This is important because it reduces errors in my programs; rather than relying on arbitrary numbers or strings, I can define clear, readable options.
For example, in the game states exercise later, using enums helped me manage transitions like ‘playing’, ‘menu’, and ‘details’ in a clean and scalable way. This showed me how enums can make larger programs easier to understand and maintain

24/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Guided Tour → Explore structs]

Explore Structs

Structs are a great way of being able to capture all the data for something (an entity) related to your program. Let’s explore how to create an use a struct within a small graphical program.

In this program we are going to show a target on the screen and record the time the user takes to “hit” this target with the mouse. We can report accuracy and average time to the user as the program runs.

program plan:

Struct	*target_data*
Fields	`x` : Distance from the left of the window
	`y` : Distance from the top of the window
	`radius` : Size of the target

Procedure	*Draw Target*
Parameters	`target` : The target to draw (a target_data value).
Description	Draws a filled circle to the screen at the target's location.

Function	*New Target*
Parameters	`x` : The x value for the target.
	`y` : The y value for the target.
	`radius` : The target's radius.
Returns	A target at the indicated location, with the indicated size.
Description	New target is used to initialise a target. The passed in parameters allow the caller to adjust the location and size of the target as they desire.

Function	*Random Target*
Returns	A target at a random location and size.
Description	Random target will return target data for a target using the current screen size as its bounds. This will ensure that the target can be drawn and seen entirely on the screen.

Function	*Mouse Over Target*
Parameters	`target` the target we are testing
Returns	True if the user has clicked, and their mouse is over the target.
Description	Check and return a boolean that indicates if the mouse is over the target.

Struct	*reaction_game_data*
Fields	`target` : The target for the user to click
	`hits` : The number of times the user has successfully clicked the target

Function	*New Reaction Game*
Returns	A new reaction game, with a random target and no hits.
Description	Use new reaction game to initialise a reaction game value. This will start with a random target and 0 hits.

Procedure	*Draw Game*
Parameters	`game` The reaction game data to draw to the screen
Description	Draw the game to the screen by clearing the screen, drawing the game's target, and then drawing game stats to the top left of the screen. This also refreshes the screen to ensure it is shown to the user.

The code:

```c
#include "splashkit.h"

struct target_data
{
    double x;
    double y;
    double radius;
};

struct reaction_game_data
{
    target_data target;
    int hits;
};

/** New target is used to initialise a target.
 * The passed in parameters allow the caller to adjust the location and size of the target as they desire
 * @param x The x value for the target.
 * @param y The y value for the target.
 * @param radius The target's radius.
 * @return A target at the indicated location, with the indicated size.
 */
target_data new_target(double x,double y, double radius)
{
    target_data target = {x,y,radius};
    return target;
}

/** Random target will return target data for a target using the current screen size as its bounds.
 * This will ensure that the target can be drawn and seen entirely on the screen.
 * @return a target at a random location and size.
 */
target_data random_target()
{
   int radius= rnd(1,300);
   return new_target(rnd(radius,800-radius),rnd(radius,600-radius),radius);
}

/**Use new reaction game to initialise a reaction game value. This will start with a random target and 0 hits.
 * A new reaction game, with a random target and no hits.
 */
reaction_game_data new_reaction_game()
{
    reaction_game_data result;
    result.target = random_target();
    result.hits =0;
    return result;
}

/** Draws a filled circle to the screen at the target's location.
 * @param target  The target to draw (a target_data value).
 */
void draw_target(target_data target)
{
    fill_circle(color_black(), target.x,target.y,target.radius);
}
```

```
/** calculate the average time per hit in the reaction game.
 * @param game The reaction game time.
 * @return The average time per hit
 */
int game_average_time(reaction_game_data game)
{
    if (game.hits == 0)
    {
        return current_ticks();
    }
    else
    {
        return current_ticks() / game.hits;
    }
}

/** Check and return a boolean that indicates if the mouse is over the target.
 * @param target the target we are testing
 * The function return True if the user has clicked, and their mouse is over the target.
 */
bool mouse_over_target(target_data target)
{
    return point_in_circle(mouse_x(),mouse_y(),target.x,target.y,target.radius);
}

```

```
/**Draw the game to the screen by clearing the screen, drawing the game's target,
 * and then drawing game stats to the top left of the screen.
 * This also refreshes the screen to ensure it is shown to the user.
 * @param game The reaction game data to draw to the screen
 */
void draw_game(reaction_game_data game)
{
    clear_screen(color_white());
    draw_text("Hits: " + std::to_string(game.hits), color_black(),0,0);
    draw_text("Average time: " + std::to_string(game_average_time(game)),color_black(),0,10);
    draw_target(game.target);
    refresh_screen();


}


int main()
```

```
int main()
{
  open_window("Reaction Timer", 800, 600);

  // Create the game data
  reaction_game_data game = new_reaction_game();

  while (!quit_requested())
  {
    process_events();

    if (mouse_clicked(LEFT_BUTTON) && mouse_over_target(game.target))
    {
      game.target = random_target();
      game.hits++;
    }

    draw_game(game);
  }

  return 0;
}
```

Output:

Through the struct exercises, I learned that structs let me group related data into a single unit. For example, in the entity manager task, instead of juggling separate variables like book_name, book_location, and book_pages, I could bundle them into one book struct. This mattered to me because it made my code cleaner, easier to expand (e.g., I could add an author field later), and more reusable. I now see how structs form the foundation for more complex data structures and will be essential as I move into larger programming projects.

24/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Guided Tour → Fly Catch]

Fly Catch

The fly catch game is going to really benefit from structs to help model the entities in the game. When we think about the game, the entities are really clear. We know that the game is made up of a spider and a fly. We can now model these as structs in our code.

My code:

```c
#include "splashkit.h"

/**
 * The fly can appear and escape (disappear), but is otherwise
 * located on the screen at a certain point.
 *
 * @field appeared  indicates if the fly is shown
 * @field x its distance from the left of the window (to the center)
 * @field y its distance from the top of the window (to the center)
 * @field appear_at_time  the time when the fly will appear
 * @field escape_at_time  the time when the fly will escape
 */
struct spider_data
{
    double x;
    double y;
};

/**
 * The spider has a location on the screen.
 *
 * @field x the distance from the left side of the window
 * @field y the distance from the top of the window
 */
struct fly_data
{
    bool appeared;
    double x;
    double y;
    int appear_at_time;
    int escape_at_time;
};
```

```c
/**
 * The game involves a spider and a fly. The player is
 * controlling the spider and the fly is trying to escape.
 *
 * @field spider  the data about the spider
 * @field fly     the data for the fly
 */
struct game_data
{
    spider_data spider;
    fly_data fly;
};


const string GAME_TIMER = "GameTimer";
const int GAME_SCREEN_WIDTH = 800;
const int SCREEN_HEIGHT = 600;
const int GAME_SCREEN_HEIGHT = SCREEN_HEIGHT - 100;
const int SPIDER_RADIUS = 25;
const int SPIDER_SPEED = 3;
const int FLY_RADIUS = 10;
```

```c
/**
 * Draw the spider on the screen
 *
 * @param spider  the spider's data
 */
void draw_spider(spider_data spider)
{
    fill_circle(color_black(), spider.x, spider.y, SPIDER_RADIUS);
}

/**
 * Draw the fly if it has appeared
 *
 * @field fly the fly data
 */
void draw_fly(fly_data fly)
{
    if (fly.appeared)
    {
        fill_circle(color_dark_green(), fly.x, fly.y, FLY_RADIUS);
    }
}
```

```
/**
 * Draw the spider and the fly to the screen.
 *
 * @param game the game data
 */
void draw_game(game_data game, int countwin, int countgame)
{
    clear_screen(color_white());
    draw_spider(game.spider);
    // generate the game counter on screen
    draw_text("You won " + std::to_string(countwin) + " out of " + std::to_string(countgame) + " games!", color_black(), "main", 6

    draw_fly(game.fly);
    refresh_screen(60);
}

/**
 * Initialise a new spider and return it to the caller.
 * The spider will be in the center of the screen.
 *
 * @return an initialised spider
 */
spider_data new_spider()
{
    spider_data spider;

    spider.x = GAME_SCREEN_WIDTH / 2;
    spider.y = GAME_SCREEN_HEIGHT / 2;
    return spider;
}
```

```
/**
 * Initialise and return new fly data. This will not have appeared,
 * but will be at a random position, and will appear within 1
 * to 3 seconds.
 *
 * @return the newly initialised fly
 */
fly_data new_fly()
{
    fly_data fly;

    fly.x = rnd(GAME_SCREEN_WIDTH);
    fly.y = 100 + rnd(GAME_SCREEN_HEIGHT);

    fly.appeared = false;
    fly.appear_at_time = 1000 + rnd(2000);
    fly.escape_at_time = 0;

    return fly;
}
```

```
int main()
{
    int countgame = 0;
    int countwin = 0;
    game_data game;

    bool flag = false;

    game.spider = new_spider();
    game.fly = new_fly();

    load_font("main", "Roboto-Light.ttf");                      // Load front for text on screen
    open_window("Fly Catch", GAME_SCREEN_WIDTH, SCREEN_HEIGHT); // create the screen

    create_timer(GAME_TIMER); // create a timer for the game to based off of.
    start_timer(GAME_TIMER);

    while (!quit_requested())
    {
        // Spider movement code
        if ((key_down(RIGHT_KEY)) && (game.spider.x + SPIDER_RADIUS < GAME_SCREEN_WIDTH)) // spider move right
        {
            game.spider.x += SPIDER_SPEED;
        }
        if ((key_down(LEFT_KEY)) && (game.spider.x - SPIDER_RADIUS > 0)) // spider move left
        {
            game.spider.x -= SPIDER_SPEED;
        }

        if ((key_down(DOWN_KEY)) && (game.spider.y + SPIDER_RADIUS < SCREEN_HEIGHT)) // spider move down
```

```
        if ((key_down(DOWN_KEY)) && (game.spider.y + SPIDER_RADIUS < SCREEN_HEIGHT)) // spider move down
        {
            game.spider.y += SPIDER_SPEED;
        }
        if ((key_down(UP_KEY)) && (game.spider.y - SPIDER_RADIUS > 100)) // spider move up
        {
            game.spider.y -= SPIDER_SPEED;
        }
        // spider movenent code end

        if (!game.fly.appeared && timer_ticks(GAME_TIMER) > game.fly.appear_at_time) // generate fly
        {
            game.fly.appeared = true;

            game.fly.x = rnd(GAME_SCREEN_WIDTH);
            game.fly.y = 100 + rnd(GAME_SCREEN_HEIGHT);
            countgame += 1;

            game.fly.escape_at_time = timer_ticks(GAME_TIMER) + 2000 + rnd(5000);
        }
        else if (game.fly.appeared && timer_ticks(GAME_TIMER) > game.fly.escape_at_time)
        {
            game.fly.appeared = false;
            game.fly.appear_at_time = timer_ticks(GAME_TIMER) + 1000 + rnd(2000);
        }
```

```
        flag = game.fly.appeared;
        if (circles_intersect(game.spider.x, game.spider.y, SPIDER_RADIUS, game.fly.x, game.fly.y, FLY_RADIUS)) // check if spider
        {
            game.fly.appeared = false;
            game.fly.appear_at_time = timer_ticks(GAME_TIMER) + 1000 + rnd(2000);
        }
        if (flag && !game.fly.appeared) // counts the number of rounds/games played
        {
            countwin += 1;
        }

        draw_game(game, countwin, countgame);

        process_events();
    }
}
```

Output:

In the fly catch program, I realized how powerful structs are in simplifying game logic. By packaging all attributes of the fly into one struct, it became easier to update and manage the program. This taught me that structs not only organize data but also improve program design when applied to real-world scenarios like games. I also learned how I could refactor older programs to take advantage of structs, which made my code cleaner and less error-prone

24/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Guided Tour → Change Calculator]

Change Calculator

The change calculator is at a reasonable place where we left it in the structuring code chapter. However, we can think through the code to see where we can make some improvements by incorporating structs and enums.

My code:

change_calculator_v3 > change_v3.cpp > ...

```cpp
#include "splashkit.h"
#include "utilities.h"

const int COIN_TYPES =6;


/**
 * The different kinds of coins in the system
 */
enum coin_type
{
    NO_COIN = -1,
    TWO_DOLLAR
    enum coin_type::FIFTY_CENTS = 2
    FIFTY_CENTS,
    TWENTY_CENTS,
    TEN_CENTS,
    FIVE_CENTS
};

/**
 * Each coin has a value and text. This is used to
 * calculate the number of coins to give in change, and
 * to determine what text to show the user.
 *
 * @field value The value of the coin
 * @field text  The coin text for output
 */
struct coin_data
{
    int value;
    coin_type kind;
    string descri;
};
```

```c
/**
 * Initialise a new coin of the indicated kind
 *
 * @param kind  The kind of coin to setup
 * @return coin data of the indicated kind
 */
coin_data new_coin(coin_type type)
{
    coin_data coin;

    coin.kind = type;

    switch(type)
    {
        case TWO_DOLLAR:
            coin.descri = "$2, ";
            coin.value = 200;
            break;
        case ONE_DOLLAR:
            coin.descri = "$1, ";
            coin.value = 100;
            break;

        case FIFTY_CENTS:
            coin.descri = "50c, ";
            coin.value = 50;
            break;
        case TWENTY_CENTS:
            coin.descri = "20c, ";
            coin.value = 20;
            break;

        case TEN_CENTS:
            coin.descri = "10c, ";
            coin.value = 10;
```

```
            case TEN_CENTS:
                coin.descri = "10c, ";
                coin.value = 10;
                break;
            case FIVE_CENTS:
                coin.descri = "5c, ";
                coin.value = 5;
                break;
            default:
                coin.descri = "ERROR";
                break;
        }
        return coin;

    }
```

```
/**
 * Give the user change of the indicated amount.
 *
 * @param change_value the amount of change to give
 */
void give_change(int change_value)
{
  int to_give;

  write("Change: ");

  for (int i = 0; i < COIN_TYPES; i++)
  {
    coin_data coin = new_coin(coin_type(i));

    to_give = change_value / coin.value;
    change_value = change_value - to_give * coin.value;
    write(std::to_string(to_give) + " x " + coin.descri);
  }

  write_line();
}
```

```
int main()
{
    int cost_of_item;
    int amount_paid;
    string again = "";

    do
    {
        cost_of_item = read_integer("Enter cost of item in cents: ");
        amount_paid = read_integer("Enter amount paid in cents: ");

        if (amount_paid >= cost_of_item)
        {
            give_change(amount_paid - cost_of_item);
        }
        else
        {
            write_line("Insufficient funds!!!");
        }

        write_line("");
        write("run again: ");
        again = read_line();

    } while (again != "N" && again != "n");
}
```

Output:

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/change_calculator_v3
$ clang++ change_v3.cpp utilities.cpp -o change_v3 -l splashkit

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/change_calculator_v3
$ ./change_v3
Enter cost of item in cents: 15
Enter amount paid in cents: 500
Change: 2 x $2, 0 x $1, 1 x 50c, 1 x 20c, 1 x 10c, 1 x 5c,

run again: y
Enter cost of item in cents: 150
Enter amount paid in cents: 100
Insufficient funds!!!

run again: y
Enter cost of item in cents: 150
Enter amount paid in cents: 250
Change: 0 x $2, 1 x $1, 0 x 50c, 0 x 20c, 0 x 10c, 0 x 5c,

run again: y
Enter cost of item in cents: 175
Enter amount paid in cents: 200
Change: 0 x $2, 0 x $1, 0 x 50c, 1 x 20c, 0 x 10c, 1 x 5c,

run again: n

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/change_calculator_v3
$
```

While working on the change calculator, I began to feel more comfortable combining structs and enums in the same program. This exercise showed me how they complement each other; structs organize related pieces of data, while enums make choices and states clearer. Together, they helped me create a program that was easier to understand and less repetitive. This gave me confidence that I could apply both concepts in future projects without hesitation

25/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Test your knowledge → Entity manager]

Output template:

```
Book entry system:

Enter the name of the book: Programmers Guide
Enter book location: https://programmers.guide
Enter number of pages: 500

Menu:
1 - Read book
2 - Print book
3 - Quit
Option: 2

Book details:
Title: Programmers Guide
Location: https://programmers.guide
Pages: 500

Menu:
1 - Read book
2 - Print book
3 - Quit
Option: 3
```

My code:

```
#include "splashkit.h"
#include "utilities.h"

/**
 * The menu_option enum represents the different choices a user can make
 * from the menu. Each option is assigned a numeric value for user input.
 */
enum menu_option
{
    READ_BOOK=1,
    PRINT_BOOK,
    QUIT
};

/**
 * The book struct stores information about a single book, including:
 * - name: the book's title
 * - location: where the book is kept
 * - num_of_pages: total number of pages
 */
struct book
{
    string name;
    string location;
    int num_of_pages;
};
```

```
/** Creates a new book by asking the user to enter details.
 *
 * @return A book struct with user-provided details.
 */
book new_book()
{
    book livre;

    write("Entre the name of the book: ");
    livre.name = read_line();
    write("Entre book location: ");
    livre.location = read_line();
    livre.num_of_pages = read_integer("Pages: ");

    return livre;
}
```

```cpp
/**Displays the menu options and reads the user's choice.
 *
 * @return A menu_option enum value based on user selection.
 */
menu_option menu()
{
    int choice;
    choice = read_integer("OPtion: ");

    switch (choice)
    {
    case 1:
        return READ_BOOK;
        break;
    case 2:
        return PRINT_BOOK;
        break;
    case 3:
        return QUIT;
        break;
    default:
        write_line("Incorect input");
        return menu();
        break;
    }
}
```

```cpp
/**
 * Prints details of the given book to the console.
 *
 * @param livre The book struct containing the details to print.
 */
void print_book(book livre)
{
    write_line("Book details: ");
    write_line("Title: " + livre.name);
    write_line("Location: " + livre.location);
    write_line("Pages: " + std::to_string(livre.num_of_pages));
}

```

```
int main()
{
    book livre;
    menu_option choice;
    write_line("Book entry system: ");
    write_line("");

    livre =  new_book();

    do
    {
        write_line("");
        write_line("Menu");
        write_line("1 - Read book");
        write_line("2 - Print book");
        write_line("3 - Quit");
        choice = menu();

        switch (choice)
        {
        case READ_BOOK:
            livre = new_book();
            break;
        case PRINT_BOOK:
            write_line("");
            print_book(livre);
            break;
        case QUIT:
        break;

        default:
            write_line("error!!!");
            break;
        }
    } while (choice != QUIT);

}
```

Output:

```
Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/entity_manager
$ clang++ entity_manager.cpp utilities.cpp -o ent -l splashkit

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/entity_manager
$ ./ent
Book entry system:

Entre the name of the book: Programmers Guide
Entre book location: https://programmers.guide
Pages: 500

Menu
1 - Read book
2 - Print book
3 - Quit
OPtion: 2

Book details:
Title: Programmers Guide
Location: https://programmers.guide
Pages: 500

Menu
1 - Read book
2 - Print book
3 - Quit
OPtion: 3

Vedant@LAPTOP-Q7B93650 MINGW64 /c/Users/Vedant/Desktop/mycode/my_c++_project/entity_manager
$
```

The entity manager task gave me practice in applying enums and structs in a larger program. I learned how structs let me manage complex data like books in a library, and enums helped handle options in a consistent way. This showed me how both tools are essential for building bigger applications, since they reduce redundancy and make the program more organized. By the end of this task, I felt I had improved not only in writing code with these tools but also in understanding how they make programs scalable

25/08/2025 - Coding[Field Guide → Part 2 → 3.Structuring data → Test your knowledge → Explore game states]

Description:

Explore Game States

Create a small program that explores different game states to allow you to switch between different logic for menus, or other screens within an interactive program.

Use an enumeration to represent the different game states: playing, showing details, or showing menu.

Have a game data struct with the current state (using your enum), the player's location (x and y), and a boolean to indicate if the game should quit.

Make sure to use functions and procedures to initialise the game data and draw the different game states. (See the hints at the end of this page for more details.)

In main, you can have all the code to update the game details. This can check the game state and then check user interaction to see what needs to change.

Loop while quit has not been requested, and the game's quit is false.

When current state is...

- **playing**:
  - Clear the screen to white
  - Draw a circle to represent the player
  - Move the player to a new random position when the user hits the space bar
  - Change the current state to:
    - showing details - when the user presses 1
    - showing menu - when the user presses 2
- **showing details**:
  - Clear the screen to light blue
  - Draw the player circle in a lighter color
  - Draw a small black circle with a radius of 2 at the player location
  - Draw the player location to the screen
  - Change the current state to playing when the user presses space
- **showing menu**:
  - Clear the screen to light green
  - Change the current state to playing when the user presses space
  - Quit the game if the user presses q

My code:

```c
#include "splashkit.h"

/** An enumeration of the different screens
 */
enum game_state
{
    PLAYING,
    SHOWING_DETAILS,
    SHOWING_MENU
};

/** composite type game_data
 * state conatin the state of the game which of the enum game_state typr
 * player_pos store the position of the player
 */
struct game_data
{
    game_state state;
    point_2d player_pos;
    bool quit;
};

/** function that returns a variable of type game_data
 * this function is used at the start the game.
 * @return A variable of type game_data
 */
game_data new_game()
{
    game_data game;
    game.state = PLAYING;
    game.quit = false;
    return game;
}
```

```c
/**procedure to create the playing screen
* @param game of type game_data
*/
void draw_playing(game_data game)
{
    clear_screen(COLOR_WHITE);
    fill_circle(COLOR_RED, game.player_pos.x, game.player_pos.y, 20);
}

/**procedure to create the details screen
 * @param game of typr game_data
*/
void draw_details(game_data game)
{
    clear_screen(COLOR_LIGHT_BLUE);
    fill_circle(COLOR_ORANGE, game.player_pos.x, game.player_pos.y, 20);
    fill_circle(COLOR_BLACK, game.player_pos.x, game.player_pos.y, 2);


    draw_text(point_to_string(game.player_pos), COLOR_BLACK, 20, 20);
}

/**procedure to create the menu screen
 *@param game of typr game_data
*/
void draw_menu(game_data game)
{
    clear_screen(COLOR_LIGHT_GREEN);
    draw_text("Press SPACE to Play, Q to Quit", COLOR_BLACK, 20, 20);
}
```

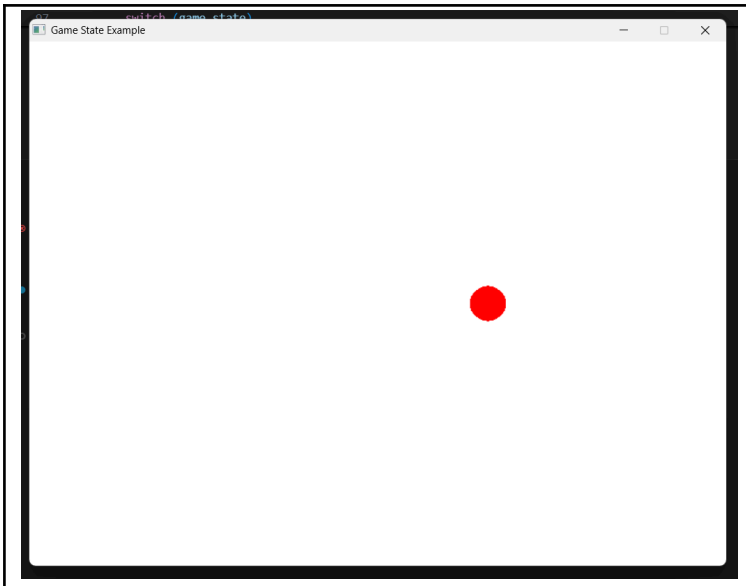
```c
/**procedure to draw screen depending on the current state
 * @param game of type game_data
 */
void draw_game(game_data game)
{
    switch (game.state)
    {
    case PLAYING:
        draw_playing(game);
        break;
    case SHOWING_DETAILS:
        draw_details(game);
        break;
    case SHOWING_MENU:
        draw_menu(game);
        break;
    }
}
```

```
int main()
{
    open_window("Game State Example", 800, 600);

    game_data game = new_game();

    while (not quit_requested() && !game.quit)
    {
        process_events();


        switch (game.state)
        {
        case PLAYING:
            if (key_typed(SPACE_KEY))
            {
                game.player_pos = random_screen_point();
            }
            else if (key_typed(NUM_1_KEY))
            {
                game.state = SHOWING_DETAILS;
            }
            else if (key_typed(NUM_2_KEY))
            {
                game.state = SHOWING_MENU;
            }
            break;
```

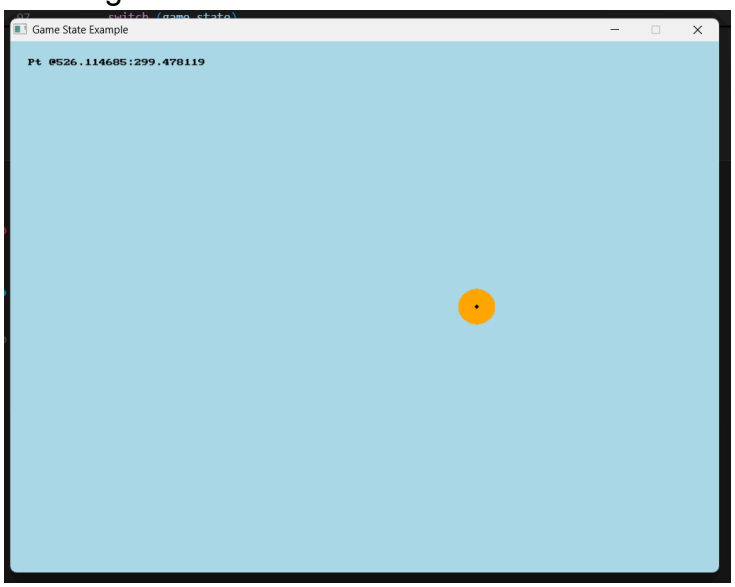
```
        case SHOWING_DETAILS:
            if (key_typed(SPACE_KEY))
            {
                game.state = PLAYING;
            }
            break;

        case SHOWING_MENU:
            if (key_typed(SPACE_KEY))
            {
                game.state = PLAYING;
            }
            else if (key_typed(Q_KEY))
            {
                game.quit = true;
            }
            break;
        }

        draw_game(game);
        refresh_screen(60);
    }

    return 0;
}
```

Output:

Playing:



Showing details:



Showing menu:

The game states exercise solidified my confidence in using enums and structs together. I learned how enums make state transitions like ‘playing’, ‘menu’, and ‘details’ much clearer, while structs store the information related to each state in an organized way. This mattered to my learning because it showed me how these concepts can be combined to manage complex program logic, such as in game development. I now feel confident that I can apply these skills to future assignments and larger projects.

Concept	Key Idea / Concept
Enum	An Enumeration allows you to create a type where the value in these variables is restricted to a list of available options.
Struct	The struct, short for structure, is a composite type whose value is made up of a number of fields.
Field	A field is an individual variable or piece of data that is part of a structure

Brief Summary of ConceptsReflection

What is the most important thing you learned from this and why?

Emun and struct. They are very useful for data manipulation, thus will always be with me for my programming journey. Thus, getting a good understanding on how they work and how to implement them is mandatory for me.

How do you see structs helping you create larger programs? Relate this to things you have created during this task.

Structs help build larger program by;

- **Grouping related data:** Instead of having separate variables scattered around like *book_name*, *book_location*, *book_pages* for program *entity manager.* The struct *book* groups them together. This makes it easier to read and manage.
- **Reusability:** once you define a struct, you can create many instances of it, like a whole library of *book.*
- **Scalability**: As programs grow, having structured data types makes it easier to add more features. For example, you could add an *author* field to the struct *book* in *entity manager* without rewriting all your code.
- **Cleaner function design:** instead of passing multiple variables to a function, you can pass just one struct, for example print_book(book livre) in program entity manager instead of passing name, location and pages separately.

Looking back at what you have learnt so far, how are you going sticking to your original learning plan?

I feel as if I've been sticking up to it pretty well. I have submitted p1 to p4 early as i had said that I wouldn't wait for the last minute to submit my assignment. As for p5 and p6, they were right on the day of the dead line(I was quite unhappy about how close it was but there was no other way).
As for what I mentioned about building a strong base for my programming journey, I feel like I am doing very well. I am doing every possible program from the field as well as creating some of my own.

How does this unit compare with other units you're currently taking?

This is one of my favourite units; not just because of the freedom to work on the assignment whenever we can but also how the unit has fully been laid out to me. I fully understand what is needed of me. Finally the field guide, this has to be the best notes/education materials that i have ever used.

How are you tracking toward achieving your target grade?

I am currently on track for HD. I have submitted all my work before the HD deadline. Thus, I feel confident that I will achieve my target grade.

References

Generative AI acknowledgement

No AI was used for either this document or any of the codes.